

Databricks SQL and Photon

**Common Patterns.
Clear Fixes.**

50 engineering patterns covering SQL warehouse sizing, the Photon engine, query federation, materialized views, query performance, SQL governance, and the interaction between Databricks SQL and Unity Catalog. Patterns every Databricks SQL engineer must recognise.

INTRODUCTION

Databricks SQL looks like a database. It is not a database.

Databricks SQL is a serverless SQL execution environment built on top of Apache Spark and accelerated by the Photon engine. Engineers who come from traditional database backgrounds bring assumptions that do not transfer. Indexes that do not exist. Statistics that behave differently. Transactions that are scoped to individual write operations rather than explicit transaction blocks. Query plans that change between runs as the optimizer adapts. The interface looks familiar. The engine is different.

This book covers seven chapters. Chapter 1 covers SQL warehouse sizing and the specific ways warehouse behavior diverges from what engineers assume, including the cost trade-off between serverless and classic compute that surfaces directly on the Azure consumption bill. Chapter 2 covers Photon and the patterns where Photon produces unexpected results or fails to deliver the expected speedup. Chapter 3 covers query federation and the performance and correctness gaps between federated queries and native queries. Chapter 4 covers materialized views and caching and the ways result caching produces stale data. Chapter 5 covers query performance tuning patterns specific to Databricks SQL. Chapter 6 covers SQL governance and the operational gaps between what users can query and what they should query. Chapter 7 covers the interaction between SQL warehouses and Unity Catalog enforcement that does not always behave identically across execution paths.

Every scenario follows the same structure: the situation, the wrong thinking, what is actually happening, and the fix. Databricks SQL problems often appear as simple slowness or unexpected costs that obscure a deeper configuration or query design issue.

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 6

Chapter 1

SQL Warehouse Sizing

Seven scenarios covering how SQL warehouse size, type, and configuration affect query performance and cost, and the specific ways warehouse behavior diverges from what engineers assume when they first configure it.

01

SQL WAREHOUSE SIZING

Your SQL warehouse is set to X-Large. Queries are no faster than on Medium.

THE SITUATION

A SQL warehouse is upgraded from Medium to X-Large to improve query performance for a slow dashboard. After the upgrade, query times are identical to Medium. The X-Large warehouse costs significantly more but delivers no performance benefit.

WRONG THINKING

Engineers assume larger warehouses run queries faster because they have more compute. An X-Large warehouse should execute queries significantly faster than a Medium.

WHAT IS ACTUALLY HAPPENING

SQL warehouse size controls the number of nodes in the warehouse cluster and the total compute available. Query performance scales with warehouse size only when the query can utilise the additional parallelism. Queries that are bottlenecked by data scan throughput on a well-laid-out Delta table, by I/O to cloud storage, or by the query logic itself may not benefit from additional nodes. A single-node Medium warehouse may be sufficient for most ad-hoc queries. Upgrading to X-Large provides benefit primarily for queries that can be parallelised across many nodes, such as large aggregations or joins on very large tables.

THE FIX

Profile the query execution plan before upgrading warehouse size. Check whether the query uses multiple nodes by examining the Spark UI task distribution. If a query runs entirely on one or two executors regardless of warehouse size, a larger warehouse will not help. For queries that are I/O-bound rather than compute-bound, improving the Delta table layout with OPTIMIZE and Z-order provides more benefit than increasing warehouse size. Reserve large warehouses for known parallel workloads and use smaller warehouses for interactive and ad-hoc queries.

02

SQL WAREHOUSE SIZING

You set auto-stop to 10 minutes. The warehouse is never warm when users arrive.

THE SITUATION

A SQL warehouse has auto-stop set to 10 minutes of inactivity. Business users who arrive in the morning find the warehouse is always stopped and spend 2 to 3 minutes waiting for it to start before running their first query. The auto-stop was intended to save cost during overnight hours.

WRONG THINKING

Engineers set a short auto-stop to minimise idle cost. 10 minutes of inactivity seems like a reasonable threshold for stopping a warehouse that is not being used.

WHAT IS ACTUALLY HAPPENING

SQL warehouse startup takes 2 to 5 minutes depending on warehouse type and size. For users who expect sub-second query response on a BI dashboard, a 3-minute warehouse startup is a significant degradation in experience. A 10-minute auto-stop means any gap in usage longer than 10 minutes results in a cold start for the next user. Morning arrivals after an overnight idle period always experience a cold start regardless of the auto-stop setting.

THE FIX

For warehouses serving business users with predictable morning usage patterns, configure a scheduled start time that brings the warehouse online 5 minutes before the first expected user arrives rather than relying on on-demand cold starts. Set auto-stop to a longer window such as 60 minutes for shared business user warehouses to avoid cold starts during normal business hours. Reserve short auto-stop settings for development or testing warehouses where cold start latency is acceptable.

03

SQL WAREHOUSE SIZING

Your serverless SQL warehouse costs 3x more than a classic warehouse for the same queries.

THE SITUATION

A SQL warehouse is migrated from classic compute to serverless compute expecting cost savings from faster startup and per-query billing. After one month, the serverless warehouse costs three times more than the classic warehouse did for the same query workload.

WRONG THINKING

Engineers switch to serverless expecting it to be cheaper due to the billing model. Serverless charges per query rather than per hour of running warehouse, which seems more efficient for intermittent usage.

WHAT IS ACTUALLY HAPPENING

Serverless SQL warehouses have a higher per-DBU rate than classic warehouses, and on Azure both the DBU charge and the underlying compute hours appear separately on the Azure consumption bill. For workloads where the warehouse runs continuously during business hours processing back-to-back queries, a classic warehouse at a lower DBU rate for the active hours, where you also have visibility into the underlying Azure VM costs, may be cheaper than serverless at a higher rate per query. The cost advantage of serverless is realised primarily for very intermittent workloads where the warehouse would otherwise be idle for significant portions of each hour. For high-utilisation warehouses, classic compute with appropriate auto-stop settings is typically more cost-effective.

THE FIX

Calculate the expected utilisation rate of the warehouse before choosing between serverless and classic. If the warehouse is expected to be actively running queries for more than 30 to 40 percent of its active hours, classic compute is likely more cost-effective. Use the Azure Cost Management views filtered by Databricks resource tags to analyse historical query patterns from a comparable classic warehouse before migrating, and validate the serverless DBU rate for your Azure region from the Azure Databricks pricing page. Monitor actual costs after the migration through Azure Cost Management and the Databricks billable usage system tables, and compare against the baseline to confirm the expected cost benefit.

04

SQL WAREHOUSE SIZING

Scaling up your warehouse did not improve concurrent query performance.

THE SITUATION

Multiple users query a SQL warehouse simultaneously. Response times are slow for concurrent users. The warehouse is scaled up from Small to Large expecting the additional capacity to handle the concurrent load. After scaling, concurrent query performance is unchanged.

WRONG THINKING

Engineers scale up the warehouse to add capacity for concurrent users. More nodes should handle more concurrent queries faster.

WHAT IS ACTUALLY HAPPENING

SQL warehouse scaling adds nodes to a single cluster. A single cluster in Databricks SQL has a maximum query concurrency limit. When that limit is reached, additional queries queue regardless of how many nodes the cluster has. The bottleneck for concurrent user workloads is often the query concurrency limit rather than per-query execution speed. Scaling up increases the resources for each individual query but does not increase the number of queries that can run simultaneously on a single cluster.

THE FIX

For concurrency problems, scale out by enabling multi-cluster warehouses rather than scaling up the warehouse size. Multi-cluster warehouses spin up additional clusters when the current cluster's concurrency limit is reached, allowing more queries to execute simultaneously. Configure the minimum and maximum cluster count based on expected peak concurrent user load. Use a Medium warehouse with multi-cluster scaling rather than an X-Large single-cluster warehouse for concurrent user workloads.

05

SQL WAREHOUSE SIZING

Your SQL warehouse is running. Queries are queuing for 5 minutes before starting.

THE SITUATION

A SQL warehouse shows a Running status. Users submitting queries experience 5-minute queue times before queries begin executing. The warehouse appears healthy and is not stopped.

WRONG THINKING

Engineers assume a Running warehouse immediately executes submitted queries. Queuing is expected to occur only when the warehouse is starting or at capacity. A running warehouse should have available compute for new queries.

WHAT IS ACTUALLY HAPPENING

SQL warehouse query queuing occurs when all concurrent query slots on all active clusters are occupied. A Running warehouse at maximum concurrency will queue new queries until existing queries complete. For a single-cluster warehouse with a concurrency limit of 10, the 11th simultaneous query queues until one of the 10 active queries finishes. The warehouse is running correctly but is at capacity. The 5-minute queue time reflects the average duration of the queries ahead in the queue.

THE FIX

Monitor concurrent query counts and queue depths using the Databricks SQL monitoring interface. If queries are consistently queuing, either enable multi-cluster auto-scaling to add additional clusters during peak load, or identify and optimise the longest-running queries that are holding concurrency slots. Set up alerts for queue depth thresholds to detect capacity problems proactively rather than reactively when users report slowness.

06

SQL WAREHOUSE SIZING

You shared one SQL warehouse across all users. One analyst's heavy query blocks everyone.

THE SITUATION

A single SQL warehouse is shared by all analysts and BI tools. An analyst runs a poorly optimised query that performs a full scan of a large table. While this query runs, all other users experience slow response times or queuing even for simple queries against small tables.

WRONG THINKING

Engineers create a single shared warehouse to simplify management and reduce cost. All users sharing one warehouse seems more efficient than maintaining separate warehouses for different user groups.

WHAT IS ACTUALLY HAPPENING

A SQL warehouse has a finite number of concurrent query slots. When a heavy query occupies multiple slots or runs for a long time, it leaves fewer slots available for other users. In a shared warehouse without query prioritisation, short simple queries from dashboard users queue behind long analytical queries from data analysts. There is no preemption mechanism that interrupts a running query to allow a faster query to proceed.

THE FIX

Separate SQL warehouses by workload type and SLA requirements. Create a dedicated warehouse for BI dashboards and interactive queries with a size appropriate for fast response times. Create a separate warehouse for analytical and heavy ad-hoc queries where longer runtimes are acceptable. This prevents heavy analytical queries from impacting dashboard user experience. Use warehouse tags to attribute costs to each workload type.

07

SQL WAREHOUSE SIZING

Your warehouse scales to maximum clusters. Query performance is still poor.

THE SITUATION

A multi-cluster SQL warehouse scales to its maximum configured number of clusters during peak load.

Despite having many clusters running, query performance is still poor and users report slow response times.

WRONG THINKING

Engineers configure multi-cluster scaling expecting it to handle any load by adding more clusters. Maximum clusters is the ceiling, and hitting it should mean sufficient capacity is available for all current queries.

WHAT IS ACTUALLY HAPPENING

Multi-cluster scaling helps with concurrency by running more queries simultaneously. It does not improve the performance of individual queries. If each query is slow due to missing table optimization, inefficient query logic, or large data scans, adding more clusters provides more concurrent slow queries rather than faster queries. The warehouse is at capacity and scaling addresses the concurrency bottleneck, but each individual query still performs poorly for reasons unrelated to concurrency.

THE FIX

Investigate individual query performance rather than warehouse scaling when queries are universally slow. Use the Databricks SQL query history to identify the slowest queries and examine their execution plans. Optimize the most expensive queries using table statistics updates, Z-order on frequently filtered columns, and query rewrites. Poor individual query performance at scale cannot be solved by adding more warehouse capacity. Fix the query and table design first, then size the warehouse for the optimized workload.

02

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 6

Chapter 2 Photon Engine

Seven scenarios covering when Photon acceleration applies, when it does not, and the specific patterns where Photon produces unexpected results or fails to deliver the expected speedup.

08

PHOTON ENGINE

You enabled Photon. Your queries are slower than before.

THE SITUATION

Photon is enabled on a SQL warehouse that was previously running without it. A set of benchmark queries that were used to establish the baseline are re-run and show longer execution times than the non-Photon baseline.

WRONG THINKING

Engineers expect Photon to make all queries faster. Photon is a performance acceleration engine so enabling it should improve or at worst maintain query performance.

WHAT IS ACTUALLY HAPPENING

Photon acceleration is most effective for columnar scans, aggregations, and joins on large datasets. For certain query patterns, the overhead of translating between Photon's native execution format and Spark's standard execution format can exceed the performance benefit. Queries that mix Photon-accelerated and non-Photon operators frequently cross the boundary between the two engines, and this boundary crossing adds overhead. Short queries on small datasets may complete faster without Photon because the startup overhead of Photon initialization is a significant fraction of total query time.

THE FIX

Benchmark queries representative of the actual production workload rather than a generic benchmark set. Photon delivers the most benefit for long-running queries on large tables with aggregation and join operations. For a workload consisting primarily of short point queries on small tables, Photon may not deliver a net benefit. Measure the 95th percentile query time for the full production workload distribution rather than focusing on a subset of benchmark queries to get an accurate picture of Photon's net impact.

09

PHOTON ENGINE

Photon is enabled. Only 10 percent of your query plan is using Photon operators.

THE SITUATION

A SQL warehouse with Photon enabled runs a complex analytical query. The query execution plan in the Spark UI shows that approximately 10 percent of the operations are using Photon operators. The remaining 90 percent uses standard Spark operators. The expected Photon speedup is not observed.

WRONG THINKING

Engineers enable Photon and assume the entire query will be accelerated. A Photon-enabled warehouse should execute all SQL operations using the Photon engine.

WHAT IS ACTUALLY HAPPENING

Photon accelerates a specific set of SQL operations including columnar scans, filters, aggregations, hash joins, and certain window functions. Operations that Photon does not yet support fall back to standard Spark execution. User-defined functions including Python UDFs, Scala UDFs, and complex lambda expressions are executed by the standard Spark engine. If the query contains UDFs, complex array operations, or other unsupported operations, only the portions of the plan that Photon supports will be accelerated. The remaining operations create boundary crossings that further reduce efficiency.

THE FIX

Review the query plan for Photon-accelerated versus non-Photon operators. Identify the specific operations preventing full Photon coverage. Replace Python UDFs with built-in SQL functions wherever possible since built-in functions are Photon-accelerated while UDFs are not. For complex transformations that require custom logic, consider whether Scala UDFs or vectorised Pandas UDFs provide better Photon compatibility than Python UDFs. Maximising the fraction of the plan that uses Photon operators provides the greatest performance benefit.

10

PHOTON ENGINE

Your Photon-accelerated query produces different results from the non-Photon version.

THE SITUATION

A query run on a Photon-enabled warehouse produces different aggregated values than the same query run on a non-Photon warehouse. The difference is in decimal precision for floating point aggregations. Some aggregated values differ in the 10th decimal place.

WRONG THINKING

Engineers assume Photon produces identical results to standard Spark for the same query. A performance engine should not change query semantics or results.

WHAT IS ACTUALLY HAPPENING

Photon uses SIMD-optimised floating point operations that may produce slightly different rounding results from Spark's standard floating point operations due to differences in the order of operations in the parallel execution. For most practical purposes these differences are negligible. However for queries that require exact decimal precision, the rounding differences between Photon and standard Spark floating point arithmetic may produce detectably different results in the last few decimal places.

THE FIX

For queries where exact decimal precision is required such as financial calculations, use DECIMAL type columns rather than DOUBLE or FLOAT. DECIMAL arithmetic in Databricks SQL is deterministic and produces consistent results regardless of the execution engine. If the existing schema uses floating point types and cannot be changed, document the expected precision tolerance for affected queries and validate results against this tolerance rather than requiring exact bit-for-bit equality between Photon and non-Photon results.

11

PHOTON ENGINE

Photon improved read performance dramatically. Write performance got worse.

THE SITUATION

Enabling Photon on a SQL warehouse significantly improves SELECT query performance as expected. However INSERT INTO, MERGE, and CREATE TABLE AS SELECT operations become slower after Photon is enabled.

WRONG THINKING

Engineers expect Photon to improve all SQL operations including writes. Since Photon accelerates SQL execution, write operations should also benefit.

WHAT IS ACTUALLY HAPPENING

Photon accelerates the read and processing phases of SQL operations. Write operations in Databricks SQL involve writing Parquet files to cloud storage, committing Delta transactions, and updating the transaction log. These write phases are not accelerated by Photon in the same way that scan and aggregation phases are. For write-heavy operations like MERGE, the read phase benefits from Photon acceleration but the write phase remains at standard speed. If Photon acceleration causes the read phase to complete significantly faster, it may increase the write phase's share of total operation time, making it appear as though writes got slower even though absolute write time is unchanged.

THE FIX

Evaluate Photon impact separately for read-heavy and write-heavy operations. For workloads dominated by reads and aggregations, Photon delivers clear benefits. For write-heavy ETL workloads, measure total operation time including both read and write phases rather than individual phase times. Optimise write performance through table layout, partition design, and write configuration independently of Photon. Photon and write performance are largely independent concerns.

12

PHOTON ENGINE

Your Python UDF is preventing Photon from accelerating the entire query.

THE SITUATION

A query that applies a Python UDF transformation in the middle of a complex analytical pipeline shows poor Photon utilisation. The UDF is applied to one column but causes the entire downstream plan to fall back to standard Spark execution.

WRONG THINKING

Engineers add a Python UDF for one specific transformation expecting the rest of the query to continue using Photon. The UDF is a small part of the overall query logic.

WHAT IS ACTUALLY HAPPENING

A Python UDF in the middle of a query plan creates a boundary that Photon cannot cross. Photon executes operations natively in C++ using columnar format. A Python UDF requires data to be converted from the columnar Photon format to row-based Python objects, passed to the Python interpreter, and the result converted back. After the UDF boundary, Photon may or may not resume acceleration for subsequent operators depending on the query plan. In many cases, a single UDF in the middle of a complex query prevents Photon from accelerating all subsequent operators, not just the UDF operation itself.

THE FIX

Replace Python UDFs with built-in Spark SQL functions wherever possible. Databricks SQL has an extensive library of built-in functions that cover most common transformation needs and all of which are Photon-compatible. For transformations that genuinely require custom logic, use Scala UDFs which have better Photon compatibility than Python UDFs, or implement the logic using SQL CASE expressions and built-in string, date, and array functions. If Python UDFs cannot be avoided, batch multiple UDF transformations into a single UDF call to minimise the number of Photon boundary crossings.

13

PHOTON ENGINE

Photon acceleration metrics show high utilisation. Wall clock time did not decrease.

THE SITUATION

A query shows high Photon operator utilisation in the query execution metrics. The Photon acceleration percentage is above 90 percent. Despite high Photon utilisation, the wall clock time for the query is the same as it was before Photon was enabled.

WRONG THINKING

Engineers monitor Photon utilisation as the primary metric for Photon effectiveness. High utilisation is assumed to mean high benefit. If Photon is doing most of the work, the query should be significantly faster.

WHAT IS ACTUALLY HAPPENING

Photon utilisation percentage measures what fraction of the query plan uses Photon operators. It does not measure how much of the total query time is spent in Photon-acceleratable operations versus other operations. If the query is dominated by operations that Photon does not accelerate such as data shuffles, network transfers, cloud storage I/O, or Delta transaction log reads, even 90 percent Photon operator coverage may not reduce total wall clock time significantly. The bottleneck is in the non-Photon phases.

THE FIX

Analyse the query execution timeline rather than just operator coverage. Identify which phases of the query execution consume the most time: data scan, shuffle, aggregation, or write. If the dominant phase is data I/O or shuffle, Photon acceleration of the computation phase provides limited benefit. Address the actual bottleneck: improve data layout for I/O-bound queries, reduce shuffle volume by optimising join strategies for shuffle-bound queries, and reduce data volume through better partition pruning.

14

PHOTON ENGINE

You disabled Photon to debug a result discrepancy. You cannot reproduce the issue without Photon.

THE SITUATION

A result discrepancy is discovered in a query running on a Photon-enabled warehouse. To isolate whether Photon is the cause, Photon is disabled and the query is re-run. The discrepancy does not reproduce without Photon. When Photon is re-enabled, the discrepancy cannot be reproduced either. The issue appears to have been intermittent.

WRONG THINKING

Engineers disable Photon as a debugging step expecting the query to reproduce the discrepancy in non-Photon mode if Photon was the cause. If the issue does not reproduce, Photon is cleared as the cause and the investigation is ended.

WHAT IS ACTUALLY HAPPENING

Intermittent query result discrepancies in Databricks SQL can be caused by data concurrency issues, stale metadata caching, or micro-batch timing rather than Photon-specific behavior. Disabling Photon changes the execution path in ways that may avoid the underlying condition that caused the original discrepancy. If the original discrepancy involved a data race where an upstream write was partially visible to the query, disabling Photon and re-running the query at a different time may produce consistent results simply because the race condition no longer exists.

THE FIX

For intermittent result discrepancies, capture the full execution context including the exact timestamp, query text, warehouse state, and upstream table versions at the time of the discrepancy. Use Delta time travel to reproduce the exact table state at the time of the original discrepancy and re-run the query against that historical version. This allows reproducible debugging independent of timing-dependent race conditions. Document whether the discrepancy reproduces with both Photon enabled and disabled against the same historical data version.

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 6

Chapter 3

Query Federation

Six scenarios covering the performance and correctness gaps between federated queries and native queries, and the specific ways federated query behavior surprises engineers who treat it as transparent SQL.

15

QUERY FEDERATION

Your federated query to PostgreSQL is 100x slower than querying PostgreSQL directly.

THE SITUATION

A federated query is set up to query a PostgreSQL database from Databricks SQL using a connection defined in Unity Catalog. The query joins a small PostgreSQL table against a large Delta table. The federated query takes 10 minutes. Running the same query directly in PostgreSQL takes 6 seconds.

WRONG THINKING

Engineers set up query federation for convenience, allowing SQL that spans both PostgreSQL and Delta tables in a single query. Federated queries are assumed to run at approximately native speed since the database executes the query.

WHAT IS ACTUALLY HAPPENING

Federated queries in Databricks SQL work by pushing predicates down to the remote database where possible, but the actual data transfer happens over a JDBC connection. For a query that joins a PostgreSQL table against a Delta table, Databricks must read the PostgreSQL table over JDBC into a Spark DataFrame and then perform the join in Spark. The 6-second PostgreSQL query may become a full table scan over JDBC rather than an optimised index-based lookup, and the data is transferred to Databricks before the join occurs. Network transfer overhead and the difference between a JDBC full scan and an optimised database execution plan explain the 100x difference.

THE FIX

Evaluate federated queries for predicate pushdown effectiveness before using them in production. Check whether filters on the federated table are pushed down to the remote database or executed in Spark after a full table transfer. For small lookup tables that are queried frequently, consider materialising the federated table as a Unity Catalog Delta table that is refreshed on a schedule rather than executing live federated queries. Live federation is most effective for real-time lookups of small filtered result sets where predicate pushdown eliminates most of the data transfer.

16

QUERY FEDERATION

Federated query predicates are not being pushed down to the source system.

THE SITUATION

A federated query includes a WHERE clause filtering on the remote table's primary key. The query is slow despite the filter. Examining the query execution plan shows that the federated table is being fully scanned and the filter is applied in Spark after transferring all rows.

WRONG THINKING

Engineers write a federated query with a filter expecting the filter to be pushed down to the source database, eliminating most of the data transfer. Predicate pushdown is assumed to work automatically for all filter conditions.

WHAT IS ACTUALLY HAPPENING

Databricks SQL query federation pushes predicates down to the remote source for a subset of filter conditions that the JDBC connector can translate to the remote database's SQL dialect. Complex predicates including functions, type casts, LIKE patterns with wildcards, or operations not supported by the remote database's SQL dialect may not be pushed down. When a predicate cannot be pushed down, the full table is transferred over JDBC and filtered in Spark. The query plan shows WHERE condition applied as a Scan Filter in Spark rather than as a condition on the JDBC read.

THE FIX

Inspect the query execution plan using EXPLAIN EXTENDED to see which predicates appear in the JDBC scan versus which are applied as Spark filters after the scan. Simplify predicate conditions to basic equality and range comparisons on indexed columns to maximise pushdown probability. If critical filters cannot be pushed down, consider materialising the filtered subset as a Delta table or using a database view on the source system that pre-filters the data before the JDBC connection reads it.

17

QUERY FEDERATION

Your federated query works from the SQL editor. Fails from a Databricks notebook.

THE SITUATION

A federated query to an external database runs successfully from the Databricks SQL editor. The same query submitted from a Databricks notebook on a job cluster fails with a connection not found or federation not supported error.

WRONG THINKING

Engineers test federated queries in the SQL editor, confirm they work, and then reference the same query in a notebook job. The query is identical so it should work in both contexts.

WHAT IS ACTUALLY HAPPENING

Query federation in Databricks SQL uses the Unity Catalog connection objects and the SQL warehouse execution context. Notebooks running on job clusters or all-purpose clusters use the Spark execution engine rather than the SQL warehouse engine. Not all federation capabilities available in Databricks SQL warehouses are available when running SQL queries through `spark.sql()` in notebook contexts. The connection objects defined in Unity Catalog may not be accessible from cluster-based compute in the same way they are from SQL warehouse compute.

THE FIX

Run federated queries exclusively through SQL warehouses using Databricks SQL or the JDBC connection to the SQL warehouse rather than through cluster-based notebook SQL. If federated data must be available in a notebook context, materialise the federated data as a Delta table first using a SQL warehouse query and then read the Delta table from the notebook. This separates the federation concern from the transformation concern and avoids context-dependent behavior.

18

QUERY FEDERATION

Query federation connection succeeded. Data types are being cast incorrectly.

THE SITUATION

A federated connection to a MySQL database returns data successfully. Comparing the values in the federated query output against the source database shows that some numeric columns have lost precision and some date columns are being returned as strings.

WRONG THINKING

Engineers validate the federated connection by confirming data is returned and the row counts match. Data type fidelity is assumed to be maintained automatically by the JDBC connector.

WHAT IS ACTUALLY HAPPENING

JDBC connectors map between the source database's type system and Spark's type system. This mapping is not always lossless. MySQL DECIMAL types may map to Spark DOUBLE with precision loss. MySQL DATETIME types may map to Spark STRING in some configurations depending on the JDBC driver version and timezone handling. The mapping depends on the specific JDBC driver version, its configuration, and how Databricks SQL negotiates types during the connection handshake.

THE FIX

Inspect the inferred schema of the federated table using DESCRIBE to see the Spark types assigned to each column. Compare against the source database schema to identify type mismatches. Override incorrect type mappings using explicit CAST expressions in the federated query or by specifying custom type mappings in the connection configuration if supported. Test all column types against known values from the source database before relying on federated data for production use.

19

QUERY FEDERATION

Your federated source has 1 billion rows. The federation query pulls all of them before filtering.

THE SITUATION

A federated query against a large operational database includes multiple WHERE conditions that should significantly reduce the rows returned. The query takes hours because the JDBC connector reads all 1 billion rows from the source database into Spark before applying the filters.

WRONG THINKING

Engineers write a federated query with appropriate filters expecting the database to handle the filtering efficiently. The query should return a small subset of rows based on the WHERE conditions.

WHAT IS ACTUALLY HAPPENING

Predicate pushdown effectiveness depends on the complexity of the filter conditions, the JDBC driver's capabilities, and whether the Databricks SQL optimizer chooses to push down the predicates. For some driver configurations or complex query structures, Databricks SQL may not push down any predicates and performs a full table scan over JDBC. For a 1-billion-row table this means transferring the entire table over the network before applying any filters in Spark.

THE FIX

For large source tables, implement server-side filtering at the source database using database views or stored procedures that pre-filter data before the JDBC connection reads it. Configure the federated connection to use a custom query that includes the filter conditions directly in the JDBC read rather than relying on predicate pushdown. In Databricks, this can be done using the `query` option in the JDBC connection configuration to push the entire filtered query to the source database.

20

QUERY FEDERATION

Query federation credentials are stored correctly. They expire and every query fails silently.

THE SITUATION

A federated connection to an external database is set up with credentials stored in a Databricks secret scope. Federated queries work correctly for months. One day all federated queries fail silently, returning no data or empty results. Investigation reveals the database credentials stored in the secret scope expired without notification.

WRONG THINKING

Engineers store credentials in a secret scope and assume they are managed and monitored by the security team. Expiry of the underlying credentials is not tracked as a platform engineering concern.

WHAT IS ACTUALLY HAPPENING

Secret scopes in Databricks store credential values as of the time they were added. If the underlying credential such as a database password or API key expires or is rotated, the secret scope continues to hold the old value. Databricks does not monitor the validity of stored credentials or alert when they become invalid. The JDBC connection may fail silently returning empty results or errors depending on how the connection handles authentication failures.

THE FIX

Implement credential expiry monitoring that alerts the platform team before database credentials stored in secret scopes expire. Set a reminder based on the credential rotation policy of the source database. Establish a credential rotation runbook that updates both the source database credentials and the Databricks secret scope value atomically. Test the federated connection after every credential rotation to confirm it succeeds before the old credentials are permanently revoked.

04

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 6

Chapter 4 Materialized Views and Caching

Seven scenarios covering materialized view freshness, refresh overhead, and the specific ways result caching produces stale data or unexpected cost increases.

21

MATERIALIZED VIEWS

Your materialized view is stale. Queries return yesterday's data without warning.

THE SITUATION

A materialized view is created to pre-compute a complex aggregation for a BI dashboard. Users report the dashboard shows data that is one day old. The materialized view refresh is configured but users were not warned that the view might be stale.

WRONG THINKING

Engineers create a materialized view to improve dashboard performance and configure a refresh schedule. The dashboard is assumed to show current data because it reads from a regularly refreshed view.

WHAT IS ACTUALLY HAPPENING

Materialized views in Databricks SQL store pre-computed query results and refresh on a defined schedule. Between refreshes, the view serves stale data. Users querying the view receive the results from the last refresh rather than current data. There is no automatic staleness warning to query consumers. If the refresh failed or was delayed, the data can be significantly older than expected without any user-visible indication.

THE FIX

Expose the materialized view's last refresh timestamp to dashboard users so they can see when the data was last updated. Add a data freshness indicator to every dashboard that uses a materialized view. Implement alerting for failed or delayed refreshes so the data engineering team is notified before users discover stale data. For dashboards with strict freshness requirements, evaluate whether the view refresh cadence is aligned with the business expectation for data currency.

22

MATERIALIZED VIEWS

You created a materialized view. It is slower to query than the base table.

THE SITUATION

A materialized view is created over a complex query expecting it to be faster than running the query against the base table. After creation, queries against the materialized view are slower than running the same query directly against the base table.

WRONG THINKING

Engineers create a materialized view expecting pre-computed results to serve instantly. A materialized view stores the results so queries against it should be much faster than recomputing from the source.

WHAT IS ACTUALLY HAPPENING

Materialized views in Databricks SQL are stored as Delta tables. Querying a materialized view is equivalent to querying the underlying Delta table that stores the pre-computed results. If this Delta table has not been optimized with OPTIMIZE and appropriate file layout, or if the materialized view stores a large result set in many small files, querying it can be slower than running the query against the well-optimized source table. The benefit of pre-computation is only realised if the stored result is efficiently queryable.

THE FIX

After creating a materialized view, check the underlying Delta table's file structure. Run **OPTIMIZE** on the materialized view's backing table to compact small files into right-sized ones through bin-packing, and add **ZORDER BY** on the columns users commonly filter on so Delta can skip irrelevant files at read time. For Unity Catalog managed tables, enable predictive optimization so that OPTIMIZE and VACUUM run automatically on a schedule appropriate to the table's update pattern. For tables with evolving query patterns, consider liquid clustering as an alternative to ZORDER. Ensure the materialized view result set is sized appropriately for the expected query patterns. If the materialized view returns millions of rows and users typically filter to a small subset, ZORDER on those columns is what enables effective file skipping.

23

MATERIALIZED VIEWS

Your materialized view refreshes every hour. The refresh is taking 55 minutes.

THE SITUATION

A materialized view is configured to refresh hourly. The refresh job is taking 55 minutes to complete. With only 5 minutes between refreshes, any delay in the refresh schedule cascades and causes the view to serve increasingly stale data.

WRONG THINKING

Engineers schedule hourly refreshes expecting each refresh to complete well within the 1-hour window. The refresh duration was not measured before setting the refresh interval.

WHAT IS ACTUALLY HAPPENING

Materialized view refresh re-executes the defining query and writes the results to the backing Delta table. For complex queries against large base tables, this refresh can be expensive. A query that takes 55 minutes to execute means the refresh barely completes before the next one is due to start. If any refresh fails or takes slightly longer than expected, the subsequent refresh is delayed or skipped, causing the view to serve data that is older than the configured refresh interval.

THE FIX

Measure the actual refresh duration before setting the refresh interval to a shorter window than the measured duration. If the refresh takes 55 minutes, an hourly schedule is not viable. Either optimize the view query to reduce refresh time, increase the refresh interval to a realistic value such as 2 to 4 hours, or evaluate whether an incremental refresh approach using Delta Lake change data feed can reduce the refresh scope to only changed rows.

24

MATERIALIZED VIEWS

A materialized view refresh failed silently. Users see data that is 3 days old.

THE SITUATION

A materialized view has been refreshing successfully for weeks. An upstream schema change causes the refresh query to fail. The refresh fails silently and users continue to see data from the last successful refresh 3 days ago. No alerts fired.

WRONG THINKING

Engineers configure the materialized view and set up a refresh schedule. Ongoing monitoring of refresh health is not implemented because the view was working correctly for an extended period.

WHAT IS ACTUALLY HAPPENING

Materialized view refresh failures do not automatically surface to dashboard users. When a refresh fails, the view continues to serve the last successfully computed result. Without explicit monitoring and alerting on refresh failures, a schema change or upstream data quality issue can cause the view to silently serve increasingly stale data indefinitely. The view appears healthy because queries succeed, but the data is not current.

THE FIX

Implement monitoring that alerts on every materialized view refresh failure immediately. Track the last successful refresh timestamp for each materialized view and alert when this timestamp exceeds the expected refresh interval by more than a defined threshold. Add a data freshness column to materialized views that shows the refresh timestamp so users can verify data currency without needing to check monitoring dashboards. Treat refresh failures as production incidents requiring immediate investigation.

25

MATERIALIZED VIEWS

You updated the base table schema. The materialized view is now broken.

THE SITUATION

A column is added to a base table that a materialized view is defined over. After the schema change, the materialized view fails to refresh with a schema mismatch error. All queries against the view also fail until the issue is resolved.

WRONG THINKING

Engineers add a column to a base table as a routine schema evolution event. The materialized view is not considered as a downstream dependency that might be affected by the schema change.

WHAT IS ACTUALLY HAPPENING

Materialized views maintain a fixed query definition that references specific columns in the base tables. When the base table schema changes in a way that affects the view's query such as renaming or dropping a column referenced in the view, the view's refresh query becomes invalid. Even when a new column is added without changing existing columns, the materialized view definition becomes stale relative to the current schema. Depending on whether the view uses `SELECT *` or explicit column lists, adding a column may or may not invalidate the view.

THE FIX

Before making schema changes to base tables, identify all materialized views and downstream dependencies that reference those tables. Update materialized view definitions to reflect schema changes before applying the change to the base table, or update them immediately after in a coordinated deployment. Use explicit column lists in materialized view definitions rather than `SELECT *` to prevent unintended behavior when new columns are added to base tables. Treat materialized views as code artifacts that require testing and deployment coordination when upstream schemas change.

26

MATERIALIZED VIEWS

Your result cache is returning stale data after an upstream table update.

THE SITUATION

A query result is cached by Databricks SQL. An upstream Delta table is updated with new data. Subsequent queries against the same SQL return cached results from before the update rather than reflecting the new data.

WRONG THINKING

Engineers expect Databricks SQL result caching to be invalidated automatically when the underlying data changes. A cache hit should only occur when the underlying data has not changed since the cache was populated.

WHAT IS ACTUALLY HAPPENING

Databricks SQL result caching stores query results for a configurable duration. Cache invalidation is triggered by Delta table version changes but the invalidation may have a delay or may not trigger for all query patterns depending on how the cache key is computed. If a query uses a function such as `CURRENT_TIMESTAMP` or `CURRENT_DATE` that should make results dynamic, the cache may still return stale results if the cache key does not account for time-dependent results.

THE FIX

For dashboards and queries where data freshness is critical, disable result caching or set a short cache TTL. Use the `NO_CACHE` query hint to bypass the cache for specific queries that must always return current data. For scheduled reports where freshness is known and bounded, configure the cache TTL to match the data refresh interval so cached results are never older than the underlying data update frequency.

27

MATERIALIZED VIEWS

You enabled disk cache on a SQL warehouse. Memory usage increased but query speed did not.

THE SITUATION

Disk cache is enabled on a SQL warehouse expecting it to speed up repeated queries against the same tables. Memory and disk utilisation increase after enabling disk cache. Query response times for repeated queries show no improvement.

WRONG THINKING

Engineers enable disk cache expecting it to cache frequently accessed table data and eliminate repeated cloud storage reads. Cached data should be served from local disk, which is faster than cloud storage.

WHAT IS ACTUALLY HAPPENING

Databricks SQL disk cache stores Parquet file data on the worker nodes' local SSDs. The cache provides benefit when the same file data is read multiple times across queries within the cache retention window. If each query reads different data or if the working set is larger than the available cache size, queries read from cloud storage regardless of whether caching is enabled. Additionally if cloud storage I/O is not the bottleneck for the queries being measured, disk cache does not improve performance even when it is being used.

THE FIX

Monitor cache hit rates to determine whether disk cache is providing benefit. A low cache hit rate indicates the working set is too large for the cache or the same data is not being accessed repeatedly. For queries on large tables where different data ranges are accessed each time, disk cache provides limited benefit. Focus disk cache tuning on frequently accessed dimension tables and reference data that is read by many queries. For fact table queries that scan different date ranges each run, disk cache hits will be low regardless of configuration.

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 6

Chapter 5

Query Performance

Eight scenarios covering the query optimization patterns specific to Databricks SQL and the specific ways standard SQL tuning intuitions do not translate to the Databricks execution model.

28

QUERY PERFORMANCE

Your SQL query runs in 2 seconds on a fresh warehouse. 20 seconds on a warm one.

THE SITUATION

A SQL query consistently runs in 2 seconds on a freshly started warehouse. When run on a warehouse that has been active for several hours processing other queries, the same query takes 20 seconds.

WRONG THINKING

Engineers expect a warm warehouse to run queries faster than a cold one since data and metadata are cached. A warm warehouse should be in an optimal state for query execution.

WHAT IS ACTUALLY HAPPENING

A warehouse that has been processing many queries may have worker JVMs in a degraded state due to accumulated garbage collection pressure, fragmented memory, or resource contention from concurrent queries. The warehouse is warm in the sense that it is running, but the JVM heap is fragmented and GC pauses are long. Additionally if other queries are consuming warehouse resources concurrently, the available parallelism for new queries is reduced. A fresh warehouse starts with clean JVM state and no resource contention.

THE FIX

Monitor JVM garbage collection metrics on warehouse nodes over time to identify whether GC degradation is the cause of warm warehouse slowdowns. Schedule warehouse restarts during low-usage periods to reset JVM state. For predictable performance, run latency-sensitive queries on a dedicated warehouse with controlled query concurrency rather than on a shared warehouse with variable load.

29

QUERY PERFORMANCE

You rewrote a query without changing logic. It is 5x faster.

THE SITUATION

A slow SQL query is rewritten by a different engineer. The new query produces identical results and is 5 times faster. Comparing the two queries side by side, the logic appears equivalent but the query structure is different.

WRONG THINKING

Engineers assume the Databricks SQL optimizer produces equivalent execution plans for logically equivalent queries. Two queries that produce the same result should take approximately the same time.

WHAT IS ACTUALLY HAPPENING

Databricks SQL uses a cost-based optimizer that produces execution plans based on the query structure and table statistics. Small structural differences between logically equivalent queries can result in very different execution plans. A join written as a subquery may produce a different plan from the same join written as a WITH clause. A filter applied before a join versus after a join may produce the same result but the optimizer cannot always recognize they are equivalent and push the filter earlier. The optimizer works with the query as written and cannot always rewrite it to its most efficient equivalent form.

THE FIX

Review the execution plan for slow queries using EXPLAIN or the Databricks SQL query profile. Identify expensive operators such as large shuffles, hash joins on large tables, or repeated table scans. Rewrite queries to help the optimizer by applying filters as early as possible, avoiding unnecessary subqueries, and using explicit join hints for large table joins where the optimizer's join strategy choice is suboptimal. Run ANALYZE TABLE to update statistics so the optimizer has accurate cardinality estimates.

30

QUERY PERFORMANCE

Your query plan shows a broadcast join on a 10GB table.

THE SITUATION

A query joins a 10GB dimension table against a 1TB fact table. The Spark UI shows the join is implemented as a broadcast join where the 10GB table is broadcast to all executors. The query runs slowly and the driver runs out of memory.

WRONG THINKING

Engineers assume the Databricks SQL optimizer chooses the most efficient join strategy based on table sizes. A broadcast join is considered efficient so the optimizer choosing it should mean good performance.

WHAT IS ACTUALLY HAPPENING

Databricks SQL has a configurable threshold for automatic broadcast join decisions, typically around 10MB by default. If the threshold has been manually set to a very high value such as 20GB, the optimizer will attempt to broadcast tables up to 20GB. Broadcasting a 10GB table requires collecting the entire 10GB on the driver node and then serialising and distributing it to all executors. The driver may run out of memory during collection and executors require significant memory to hold the broadcast copy. The broadcast join is triggered by a misconfigured threshold rather than an informed decision about whether broadcasting is beneficial.

THE FIX

Check the `spark.sql.autoBroadcastJoinThreshold` setting and reset it to a reasonable value such as 10MB which is the Databricks default. For large dimension tables that genuinely benefit from broadcast joins due to their access patterns, use explicit broadcast join hints rather than raising the global threshold. Use `SELECT /*+ BROADCAST(dimension_table) */ ...` only for tables small enough to fit comfortably in driver and executor memory.

31

QUERY PERFORMANCE

Adding a filter to your query made it slower.

THE SITUATION

A query without a WHERE clause runs in 30 seconds. Adding a WHERE clause that filters to approximately 1 percent of rows makes the query take 45 seconds.

WRONG THINKING

Engineers expect filters to always make queries faster by reducing the data processed. Filtering to 1 percent of rows should reduce work by 99 percent.

WHAT IS ACTUALLY HAPPENING

Adding a filter can change the query execution plan in ways that offset the benefit of reduced data volume. A filter that prevents a hash join from using a broadcast strategy may force a more expensive sort-merge join. A filter on a non-Z-ordered column requires a full table scan to evaluate, adding scan overhead without enabling file skipping. Additionally if the filter creates data skew where 99 percent of traffic goes to 1 executor handling the 1 percent of matching rows, the overall job time is dominated by the single skewed executor rather than the 99 percent that finished quickly.

THE FIX

Examine the execution plan before and after adding the filter to identify how the plan changed. If the filter changed the join strategy to a more expensive one, add an explicit join hint to restore the preferred strategy. If the filter does not enable file skipping, consider whether adding Z-order on the filtered column would allow Delta to skip non-matching files. For filters that create severe skew, apply salting to the affected partition key to distribute the skewed workload across multiple executors.

32

QUERY PERFORMANCE

Your aggregation query scans 10TB to return 10 rows.

THE SITUATION

A Databricks SQL aggregation query returns 10 rows representing summary statistics. The query history shows it scanned 10TB of data to produce those 10 rows. The query takes 20 minutes.

WRONG THINKING

Engineers assume aggregation queries are inherently efficient because they return few rows. A query returning 10 rows should be fast regardless of the underlying data volume.

WHAT IS ACTUALLY HAPPENING

Aggregation queries must scan all rows that contribute to the aggregation regardless of how few rows appear in the final output. A `COUNT(*)` or `SUM()` over a 10TB table must read all 10TB to compute the result. The output size has no bearing on the scan volume. Without effective partition pruning or file skipping, the query must scan all data. If the aggregation does not have a `WHERE` clause that maps to a partition or Z-order column, no optimization can reduce the scan below the full table size.

THE FIX

Apply partition pruning by adding date or category filters that reduce the scan scope to the relevant partitions. If historical aggregations are needed repeatedly, pre-compute and store them in a smaller summary table rather than recomputing from the full 10TB table on every query. For ad-hoc aggregations that genuinely require scanning all data, accept the scan cost and right-size the warehouse for the expected data volume rather than trying to avoid the scan.

33

QUERY PERFORMANCE

Two identical queries from different users return at different speeds.

THE SITUATION

Two analysts run the exact same SQL query at different times of day. One analyst's query returns in 5 seconds. The other analyst's identical query takes 2 minutes. Both use the same warehouse.

WRONG THINKING

Engineers assume identical queries on the same warehouse always run at the same speed. Query performance is treated as deterministic for a given warehouse and query.

WHAT IS ACTUALLY HAPPENING

Query performance on a shared SQL warehouse is affected by warehouse state at the time of execution. If the first query runs when the warehouse is lightly loaded and has cached metadata, it benefits from cache hits and full parallelism. The second query may run when the warehouse is handling many concurrent queries, causing resource contention and reduced parallelism. Additionally the Delta table statistics used by the optimizer may have changed between the two runs if ANALYZE TABLE was run, producing a different execution plan. Cloud storage throughput varies with time of day affecting I/O-bound queries.

THE FIX

For predictable query performance, use a dedicated warehouse with controlled concurrency rather than a shared warehouse. Monitor query execution context including concurrent queries and warehouse resource utilisation at the time of each query to understand performance variation. For benchmarking and SLA measurement, run queries under controlled conditions with known warehouse load rather than comparing times across different warehouse states.

34

QUERY PERFORMANCE

Your query uses a view. The underlying table is fully scanned despite filters on the view.

THE SITUATION

A view is created over a large Delta table with a complex query. A user queries the view with a WHERE clause filter. The query profile shows the underlying table is fully scanned despite the filter.

WRONG THINKING

Engineers create a view with a WHERE clause and expect queries against the view that add additional filters to benefit from both the view's filter and the query's filter, limiting the scan scope.

WHAT IS ACTUALLY HAPPENING

Views in Databricks SQL are logical objects that are expanded into the underlying query at execution time. The combined query including the view definition and the user's additional filters is submitted to the optimizer as a single query. If the optimizer cannot push the combined filters down to the Delta file scanning layer, the table is fully scanned. This can happen when the view definition contains expressions that prevent predicate pushdown such as complex transformations, non-deterministic functions, or subqueries that the optimizer cannot simplify.

THE FIX

Examine the full expanded query plan for view-based queries to see which predicates appear at the Delta scan level versus which are applied as post-scan filters. Simplify view definitions to contain only column selections and simple transformations that allow the optimizer to push filters through the view to the underlying table. If a view contains operations that prevent predicate pushdown, consider materialising the view as a materialized view or a pre-computed Delta table for performance-sensitive query patterns.

35

QUERY PERFORMANCE

Statistics are up to date. The query planner is still making the wrong join choice.

THE SITUATION

Table statistics are refreshed using `ANALYZE TABLE` before running a complex query. Despite current statistics, the query planner chooses a sort-merge join instead of a broadcast join for a small dimension table, producing a slow query.

WRONG THINKING

Engineers run `ANALYZE TABLE` and assume the optimizer will make optimal decisions with current statistics. Up-to-date statistics should produce the best execution plan.

WHAT IS ACTUALLY HAPPENING

Table statistics provide cardinality and distribution information but the optimizer's join strategy decision also depends on available memory, the configured thresholds, and other query factors. Statistics may be accurate but if the estimated broadcast size exceeds the `autoBroadcastJoinThreshold` setting, the optimizer will not choose broadcast join regardless of whether broadcasting would be faster. Additionally statistics are computed at the column level and the optimizer may not have accurate statistics for derived expressions or complex join conditions.

THE FIX

When current statistics do not produce the expected join strategy, use explicit join hints to override the optimizer's choice. Add `/*+ BROADCAST(small_table) */` to force a broadcast join for a known-small table. Use `/*+ MERGE(large_table, large_table2) */` to force sort-merge join when the optimizer incorrectly chooses broadcast for large tables. Document join hints used for performance-critical queries so they are maintained when the query is modified.

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 6

Chapter 6

SQL Governance

Eight scenarios covering the operational gaps between what users can query and what they should query, and the specific ways ungoverned SQL access creates cost, security, and freshness problems.

36

SQL GOVERNANCE

A user ran `SELECT *` on a 10TB table from the SQL editor. It ran to completion.

THE SITUATION

An analyst opens the Databricks SQL editor and runs `SELECT * FROM large_fact_table` without any filter. The query scans 10TB and runs for 45 minutes before completing. The analyst wanted to preview the data but did not realise the query would return all rows.

WRONG THINKING

Engineers assume analysts understand the cost implications of unfiltered queries on large tables. The SQL editor is treated as a tool for experienced users who know to add LIMIT clauses for exploratory queries.

WHAT IS ACTUALLY HAPPENING

Databricks SQL does not impose a default row limit or query timeout on queries submitted through the SQL editor unless explicitly configured. An unfiltered SELECT on a large table runs to completion consuming significant warehouse capacity and DBU cost. For organisations with many analysts of varying SQL experience levels, unfiltered large table queries are a common and expensive mistake.

THE FIX

Configure a maximum bytes scanned limit or a query execution timeout on SQL warehouses used by analysts. Set a default LIMIT for SQL editor sessions using warehouse configuration. Add a governance policy that flags or blocks queries that scan above a defined data volume threshold without a WHERE clause. Educate analysts on the cost implications of unfiltered queries and provide sample query templates that include appropriate filters and LIMIT clauses for exploratory work.

37

SQL GOVERNANCE

Your query history shows a query that nobody on the team recognises running.

THE SITUATION

The Databricks SQL query history shows a query running at 2am against a sensitive table. The query is attributed to a service account. No team member recalls scheduling this query or recognises the SQL.

WRONG THINKING

Engineers review query history when investigating a data access incident. Unrecognised queries are assumed to be from a known automated process that was not documented.

WHAT IS ACTUALLY HAPPENING

Databricks SQL query history records all queries including those run by service principals, automated tools, and scheduled jobs. If a service principal's credentials were leaked or if an undocumented process was created by a former team member, queries can appear in the history with no current team member able to explain them. An unexplained 2am query against a sensitive table is a potential security incident regardless of whether it has a benign explanation.

THE FIX

Treat unrecognised queries in the history as security incidents until explained. Identify the service principal that ran the query and trace all jobs and applications using that service principal. Rotate the service principal's credentials immediately if the access cannot be explained. Implement a regular audit of SQL query history for sensitive tables to identify unexpected access patterns. Require all automated SQL access to use service principals with documented purposes and revoke service principals when their associated workloads are decommissioned.

38

SQL GOVERNANCE

You set a query timeout. Long running queries are still completing past the limit.

THE SITUATION

A query timeout of 5 minutes is configured on a SQL warehouse to prevent runaway queries. Some queries that run for 10 to 15 minutes still complete rather than being terminated at the 5-minute limit.

WRONG THINKING

Engineers configure the timeout expecting it to act as a hard limit that terminates any query exceeding the configured duration.

WHAT IS ACTUALLY HAPPENING

Query timeout configuration in Databricks SQL warehouses may apply differently depending on the query source and execution context. Timeouts configured at the warehouse level apply to queries submitted through the warehouse's SQL endpoint. Queries submitted through different paths such as the notebook SQL context or direct JDBC connections may have different timeout settings. Additionally timeout enforcement has a granularity where the query is checked at completion of each task stage rather than continuously, meaning a query may complete a long-running final stage before the timeout check fires.

THE FIX

Verify the exact timeout configuration path and confirm it applies to all query submission methods used in the warehouse. Test timeout behavior by deliberately submitting a known long-running query and confirming it is terminated at the expected time. For queries submitted through JDBC, configure the query timeout at the JDBC connection level in addition to the warehouse configuration. Monitor query durations in the query history and alert on queries exceeding the timeout threshold to identify cases where the timeout is not enforcing correctly.

39

SQL GOVERNANCE

Your SQL alerts are configured. They fired 6 hours after the threshold was crossed.

THE SITUATION

SQL alerts are configured on a key metric query to fire when a threshold is crossed. The threshold is crossed at 9am. The alert notification arrives at 3pm, 6 hours after the threshold was crossed.

WRONG THINKING

Engineers configure SQL alerts expecting near-real-time notification when thresholds are crossed. An alert that fires 6 hours late provides no operational value for time-sensitive incidents.

WHAT IS ACTUALLY HAPPENING

Databricks SQL alerts check the condition based on the schedule configured for the alert, not continuously. If the alert is scheduled to check every 6 hours, it will not detect a threshold crossing until the next scheduled check after the crossing occurs. The alert fires based on the result of the next query execution after the threshold was crossed, not at the moment of crossing. Alerts with infrequent check intervals provide significantly delayed notification.

THE FIX

Set the alert check frequency to match the required notification latency. For near-real-time alerts, configure the alert to run every 1 to 5 minutes. Evaluate the cost of frequent alert query execution against the value of faster notification. For critical operational metrics, use more frequent checks. For business metrics where hourly notification is acceptable, less frequent checks reduce unnecessary query cost. Consider whether the underlying data refresh frequency supports frequent alert checks, since an alert that checks every minute against hourly-refreshed data provides no benefit over an hourly check.

40

SQL GOVERNANCE

A dashboard is refreshing every minute. It is consuming 80 percent of warehouse capacity.

THE SITUATION

A Databricks SQL dashboard is configured to auto-refresh every minute. This single dashboard is consuming 80 percent of the warehouse's query concurrency slots, leaving other users with degraded performance.

WRONG THINKING

Engineers configure a frequent auto-refresh to ensure the dashboard always shows current data. The impact of frequent refreshes on warehouse capacity is not calculated before setting the refresh interval.

WHAT IS ACTUALLY HAPPENING

A dashboard with multiple charts each executing a separate SQL query will submit one query per chart on every refresh. A dashboard with 10 charts refreshing every minute submits 10 queries per minute or 600 queries per hour from a single dashboard. On a warehouse with limited concurrency, this continuous stream of dashboard queries occupies a significant fraction of available query slots. Other users attempting to run ad-hoc queries queue behind the dashboard's automated query stream.

THE FIX

Set dashboard auto-refresh intervals based on the actual data update frequency rather than the minimum possible interval. If the underlying data refreshes hourly, a 1-minute dashboard refresh provides no additional data freshness. Match the dashboard refresh interval to the data update cadence. For dashboards that must show current data, ensure the underlying tables are updated frequently rather than setting aggressive refresh intervals that scan stale data repeatedly. Monitor dashboard-generated query volume as a separate category in the warehouse query history.

41

SQL GOVERNANCE

You granted dashboard view access. Users can see the underlying SQL queries.

THE SITUATION

A dashboard is shared with business users who have view-only access. The business users discover they can see the underlying SQL queries for each chart by clicking the chart menu or by accessing the dashboard edit mode through an alternative URL path.

WRONG THINKING

Engineers share dashboards with view access expecting business users to see only the visualised output. View access is assumed to hide the underlying query logic from non-technical users.

WHAT IS ACTUALLY HAPPENING

Databricks SQL dashboard sharing with view access restricts the ability to edit dashboards but may not fully restrict access to the underlying query text depending on the sharing configuration and the user's technical knowledge. Users who navigate to the underlying query editor or who know the direct URL to the query can sometimes access the query text associated with dashboard visualisations. For dashboards that contain proprietary business logic or sensitive filter conditions in the SQL, this exposure may be unintended.

THE FIX

Review what information is exposed to view-only dashboard users by testing access as a view-only user. For dashboards where the query logic must be hidden, use materialized views or pre-computed tables as the data source rather than embedding complex logic in the dashboard query. Query the simplified output table from the dashboard to decouple the business logic from the dashboard layer. Apply Unity Catalog permissions to restrict access to the underlying tables so that even if the query text is visible, users without table access cannot run the queries independently.

42

SQL GOVERNANCE

Your scheduled SQL query is failing silently. The dashboard shows the last successful result.

THE SITUATION

A scheduled SQL query that refreshes a dashboard dataset fails due to an upstream schema change. The dashboard continues to display the results from the last successful execution without any visible error indication. Business users report the dashboard as current when it is actually showing stale data.

WRONG THINKING

Engineers assume that if a dashboard query fails, the dashboard will show an error state visible to users. A failing query is expected to be clearly indicated rather than silently showing stale results.

WHAT IS ACTUALLY HAPPENING

Databricks SQL dashboards display the cached results from the last successful query execution when the current execution fails. The dashboard appears normal to viewers because it still shows data rather than an error. Alert notifications for failed scheduled queries go to the configured notification recipients, which may be the data engineering team rather than the business users viewing the dashboard. Business users have no visibility into the query failure unless they check the query execution history.

THE FIX

Add a last-updated timestamp to all dashboards sourced from scheduled queries so users can verify data currency. Configure scheduled query failure alerts to also notify the dashboard's primary business users, not only the data engineering team. Implement a staleness detection query that checks whether the dashboard data has been updated within the expected window and displays a prominent warning on the dashboard when data is stale. Treat dashboard data staleness as a user-visible incident rather than a backend monitoring concern.

43

SQL GOVERNANCE

A user exported a dashboard to PDF. It included data they should not have access to.

THE SITUATION

A business user exports a Databricks SQL dashboard to PDF for a presentation. The PDF includes data from a section of the dashboard that shows aggregated metrics from a sensitive table. The user should have access to the aggregated metrics but the PDF export includes underlying detail rows that were not intended to be visible.

WRONG THINKING

Engineers design the dashboard to show only appropriate aggregated metrics. The export functionality is assumed to produce the same view that users see in the browser, which shows only permitted data.

WHAT IS ACTUALLY HAPPENING

Dashboard PDF exports may include more data than is visible in the browser view depending on how the export is implemented. If visualisations truncate data in the browser for display purposes but the export renders the full underlying dataset, the PDF may contain more rows than the user saw on screen. Additionally if different sections of the dashboard have different access controls and the export collects all sections together, the PDF may include data from sections the user had access to but was not intended to receive in export form.

THE FIX

Test the PDF export functionality explicitly as part of the dashboard governance review. Verify that the exported PDF contains only data appropriate for the viewing user. For dashboards with mixed-sensitivity sections, consider splitting them into separate dashboards with appropriate access controls rather than using section-level visibility within a single dashboard. Restrict the export functionality to users with explicit export permissions rather than making it available to all view-access users.

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 6

Chapter 7

SQL and Unity Catalog

Seven scenarios covering the specific ways queries that work in one Databricks context fail in another, and the interaction between SQL warehouses and Unity Catalog enforcement that does not always behave identically across execution paths.

44

SQL AND UNITY CATALOG

Your SQL query works in a notebook. Fails in the SQL editor with a different error.

THE SITUATION

A SQL query runs correctly in a Databricks notebook using `spark.sql()`. The same query submitted to the Databricks SQL editor on the same warehouse fails with a syntax or object not found error.

WRONG THINKING

Engineers test SQL in notebooks and assume it will run identically in the SQL editor since both use SQL against the same tables.

WHAT IS ACTUALLY HAPPENING

Notebooks running with `spark.sql()` use the Spark SQL dialect which has some differences from the ANSI SQL dialect used by Databricks SQL warehouses. Certain Spark SQL syntax such as backtick-quoted identifiers, some function names, or certain implicit type coercions work in Spark SQL but differ in the ANSI SQL mode used by Databricks SQL. Additionally notebooks may have a different default catalog or database context than the SQL editor, causing object not found errors when the same two-level reference resolves differently.

THE FIX

Test SQL queries in the Databricks SQL editor rather than notebooks for all queries intended for SQL warehouse execution. Use ANSI SQL-compliant syntax and three-level namespace references. When moving SQL from notebooks to the SQL editor, review all function calls and syntax patterns against the ANSI SQL standard. Set the default catalog explicitly in the SQL editor session to match the namespace used in notebook testing.

45

SQL AND UNITY CATALOG

Row filters applied in Unity Catalog are not being enforced in SQL editor queries.

THE SITUATION

Row filters are configured in Unity Catalog on a sensitive table. Interactive queries from notebooks on Unity Catalog-enabled clusters correctly show filtered results. Queries against the same table from the Databricks SQL editor return unfiltered results.

WRONG THINKING

Engineers configure Unity Catalog row filters and validate them in notebook queries. The SQL editor is assumed to enforce the same row filters since it uses the same Unity Catalog tables.

WHAT IS ACTUALLY HAPPENING

Unity Catalog row filter enforcement depends on the cluster access mode. Notebooks on clusters configured for Unity Catalog single-user or shared access mode enforce row filters. SQL warehouses also enforce Unity Catalog row filters. However if the SQL editor is connecting to a classic SQL warehouse that is not fully configured for Unity Catalog governance, row filters may not be applied. Additionally the identity used for SQL editor queries must match the identity for which row filter rules are defined.

THE FIX

Verify the SQL warehouse type and configuration supports Unity Catalog row filter enforcement. Serverless SQL warehouses and pro SQL warehouses enforce Unity Catalog governance including row filters. Classic SQL warehouses may require specific configuration. Check the Unity Catalog system tables to confirm row filter assignments are active and that the current user's identity matches the filter conditions. Test row filter enforcement in the SQL editor explicitly as a different step from testing in notebooks.

46

SQL AND UNITY CATALOG

Your SQL warehouse cannot see Unity Catalog tables that a cluster can see.

THE SITUATION

Tables created in Unity Catalog are visible and queryable from a notebook on a Unity Catalog-enabled cluster. The same tables are not visible when browsing the catalog in the Databricks SQL editor connected to a SQL warehouse.

WRONG THINKING

Engineers create Unity Catalog tables and confirm they are accessible from compute. The SQL editor is assumed to provide access to all Unity Catalog tables since it uses the same metastore.

WHAT IS ACTUALLY HAPPENING

SQL warehouse access to Unity Catalog tables depends on the workspace configuration and the SQL warehouse's metastore binding. If the workspace has not been fully configured to use Unity Catalog as the default metastore for SQL warehouses, the SQL editor may connect to the legacy Hive metastore rather than Unity Catalog. Additionally the SQL warehouse must be attached to the Unity Catalog metastore and the user must have appropriate Unity Catalog grants for the tables to be visible.

THE FIX

Verify the SQL warehouse is configured to use Unity Catalog by checking the workspace admin settings for the default metastore assignment. In the SQL editor, run `SELECT current_metastore()` to confirm which metastore is active. If the Hive metastore is active instead of Unity Catalog, reconfigure the workspace to set Unity Catalog as the default metastore for SQL warehouses. Ensure the SQL warehouse's service principal has the required Unity Catalog grants for the tables that should be visible.

47

SQL AND UNITY CATALOG

You created a view in Unity Catalog from the SQL editor. Notebooks cannot query it.

THE SITUATION

A view is created from the Databricks SQL editor and confirmed to exist in Unity Catalog. When a notebook attempts to query the same view using `spark.sql()`, it fails with a table or view not found error even though the notebook is on a Unity Catalog-enabled cluster.

WRONG THINKING

Engineers create a view in the SQL editor and assume it is universally accessible from any Unity Catalog-enabled compute. A view in Unity Catalog should be queryable from any context with appropriate permissions.

WHAT IS ACTUALLY HAPPENING

Views created from the Databricks SQL editor are created in the context of the SQL warehouse's active catalog and schema. If the SQL editor has a different default catalog or schema than the notebook's context, the view exists in the SQL editor's catalog but the notebook is looking in its own default catalog. Additionally views created through SQL warehouses use the definer's SQL warehouse identity, which may differ from the cluster identity when the view is accessed from a notebook.

THE FIX

Always use fully qualified three-level namespace references when creating and querying views across different execution contexts. Create the view using `CREATE VIEW catalog.schema.view_name AS ...` and query it using `spark.sql('SELECT * FROM catalog.schema.view_name')` to ensure the same object is referenced regardless of the session's default catalog and schema settings.

48

SQL AND UNITY CATALOG

Your SQL warehouse is using Hive metastore tables instead of Unity Catalog tables.

THE SITUATION

After enabling Unity Catalog, queries in the SQL editor against two-level namespace table references return data from the Hive metastore rather than Unity Catalog. Unity Catalog tables with the same schema and table name exist but are not being used.

WRONG THINKING

Engineers enable Unity Catalog and expect all two-level namespace references to resolve to Unity Catalog. The Hive metastore is assumed to be superseded by Unity Catalog once it is enabled.

WHAT IS ACTUALLY HAPPENING

Enabling Unity Catalog does not automatically redirect existing two-level namespace references from the Hive metastore to Unity Catalog. The default catalog for SQL warehouse sessions must be explicitly configured to determine where two-level references resolve. If the default catalog is set to `hive_metastore`, two-level references still resolve to the Hive metastore. If the default catalog is a Unity Catalog catalog name, two-level references resolve there instead.

THE FIX

Configure the default catalog for SQL warehouse sessions to the Unity Catalog catalog that should be used for two-level reference resolution. Set this at the warehouse level using the workspace admin SQL warehouse configuration. Verify the effective default catalog for new sessions by running `SELECT current_catalog()` in the SQL editor. After confirming the default catalog is correct, audit and update any dashboards or queries that use two-level namespace references to ensure they resolve to the intended Unity Catalog tables.

49

SQL AND UNITY CATALOG

Column masking works from notebooks. Is bypassed from the SQL editor.

THE SITUATION

Column masking is applied to a sensitive column in Unity Catalog. Notebook queries on Unity Catalog-enabled clusters correctly show masked values. The same query from the Databricks SQL editor returns unmasked values.

WRONG THINKING

Engineers validate column masking in notebook queries and confirm it works. The SQL editor is assumed to enforce the same masking since it accesses the same Unity Catalog tables.

WHAT IS ACTUALLY HAPPENING

Column masking enforcement in Databricks SQL depends on the SQL warehouse configuration and whether it is a Unity Catalog-aware warehouse type. Classic SQL warehouses that predate full Unity Catalog integration may not enforce column masks applied through Unity Catalog. The notebook on a Unity Catalog-enabled cluster correctly enforces masking but the SQL warehouse, if not fully configured for Unity Catalog governance, may bypass the masking layer and return raw values.

THE FIX

Use serverless or pro SQL warehouses for all queries against tables with Unity Catalog column masks as these warehouse types have full Unity Catalog governance integration. Verify column masking enforcement on the SQL warehouse explicitly by testing with a non-privileged user identity. Do not assume that validation in a notebook context confirms enforcement in a SQL warehouse context, as the two execution environments have different levels of Unity Catalog integration depending on their configuration.

50

SQL AND UNITY CATALOG

Your SQL warehouse service principal has the wrong Unity Catalog permissions. All dashboards broke.

THE SITUATION

A SQL warehouse service principal's Unity Catalog permissions are modified during a permission audit. Immediately after the modification, all scheduled queries and dashboard refreshes running on that warehouse begin failing with permission denied errors.

WRONG THINKING

Engineers modify service principal permissions as part of a routine governance review without assessing which warehouse relies on that service principal. Permission changes are assumed to be low-risk administrative operations.

WHAT IS ACTUALLY HAPPENING

SQL warehouses use a service principal for executing queries when users are not directly authenticated, such as for scheduled queries and embedding scenarios. If this service principal's Unity Catalog permissions are reduced below what is required for the queries it executes, all queries running under that identity fail. A single service principal serving a shared warehouse means that a permission change to that principal affects all scheduled queries and dashboards running on the warehouse simultaneously.

THE FIX

Document which SQL warehouses use each service principal and what Unity Catalog permissions are required for the queries those warehouses execute. Before modifying a service principal's permissions, assess which warehouses and scheduled queries will be affected. Test permission changes in a staging environment first. For production warehouses serving many dashboards, require a change review process before modifying the warehouse service principal's permissions. Use separate service principals for different warehouses to limit the blast radius of any individual permission change.

CONCLUSION

A running warehouse is not a fast warehouse.

Across 50 scenarios the pattern repeats. Larger warehouses do not make single-threaded queries faster. Photon does not accelerate operations that touch UDFs or write paths. Materialized views serve stale data when their refresh fails. Federated queries pull entire tables across the network when predicates cannot be pushed down. Result cache returns yesterday's answer to a question asked today. SQL governance configured at one layer is bypassed at another. The lesson is that Databricks SQL is a stack of independent execution paths, each with its own optimization model and enforcement boundary, and the running status of a warehouse tells you almost nothing about whether queries against it are correct, current, or fast.

ALSO IN THE SERIES

Clusters and Compute

▮ Your job failed in 30 seconds. The cluster spent 28 of them starting up.

Delta Lake

▮ You ran OPTIMIZE. Your queries got slower.

Unity Catalog

▮ You created an external location. Users can still access the storage directly.

And more. Download the full series free at ssanjaychandra.com

A NOTE ON THIS RELEASE

This is Book 6 in the Azure Databricks Engineering Patterns series. Databricks SQL warehouse semantics, Photon engine behavior, federation, materialized views, and query optimization are largely the same on every cloud, so almost every scenario in this book is identical to the cloud-neutral original. The one place Azure shows up is the cost trade-off between serverless and classic SQL warehouses, which is visible on the Azure consumption bill and through Azure Cost Management. Databricks and Azure both evolve quickly and some guidance, particularly around Photon operator coverage, may need updating as the platforms mature. If you spot a technical inaccuracy or a scenario that deserves deeper treatment, your feedback is genuinely welcome.

ACKNOWLEDGEMENTS

Claude (Anthropic) was used as a writing assistant while putting this book together.

CONNECT



www.ssanjaychandra.com